Сравнение алгоритмов поиска дубликатов

Аннотация. Рассматриваются попарное сравнение элементов и поиск повторов с помощью множества. Для небольших массивов оба подхода дают одинаковый результат, но число операций различается. Цель работы — показать зависимость числа сравнений от размера массива.

Ключевые слова: алгоритмы, дубликаты, сложность вычислений.

Метод. Попарный алгоритм сравнивает каждый элемент со всеми последующими. Для массива без повторов число сравнений составляет $L = n(n - 1) / 2$. Алгоритм с множеством последовательно проверяет наличие элемента и добавляет его в множество; при стандартных предположениях о хешировании ожидаемое время линейно.

Результаты. Для массивов из 10, 20, 40 и 80 различных элементов попарный алгоритм выполняет соответственно 45, 190, 780 и 3160 сравнений. Эти значения получены по формуле и не являются измерениями времени работы.

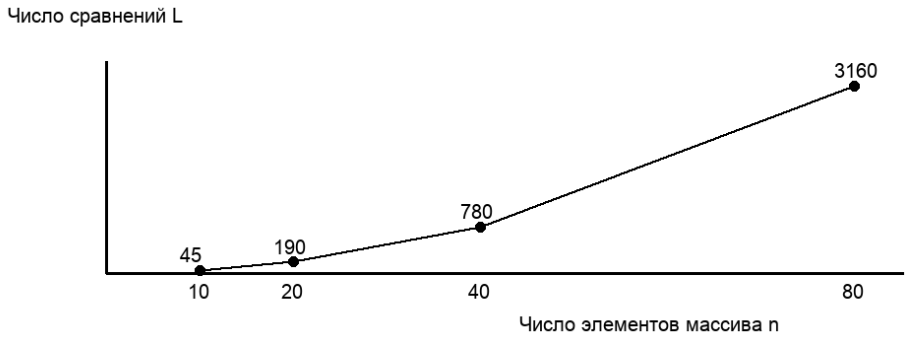

Рисунок 1. Расчётное число сравнений L в массиве из n различных элементов. Время работы не измерялось.

Заключение. Попарное сравнение требует квадратичного числа операций. Для больших массивов применение множества сокращает ожидаемую трудоёмкость. Для воспроизводимого измерения времени необходимо дополнительно зафиксировать реализацию и среду исполнения.

Описание алгоритмов и пример

Оба алгоритма отвечают на вопрос, есть ли в массиве хотя бы два равных элемента. Они возвращают логическое значение и завершаются сразу после обнаружения повтора.

Попарный алгоритм

```
для i от 0 до n − 2:
    для j от i + 1 до n − 1:
        если A[i] = A[j]: вернуть Истина
вернуть Ложь
```

Алгоритм с множеством

```
S := пустое множество
для каждого x в A:
    если x содержится в S: вернуть Истина
    добавить x в S
вернуть Ложь
```

Для массива [3, 1, 3, 2] оба алгоритма возвращают Истина. Попарный алгоритм обнаруживает совпадение элементов с индексами 0 и 2; алгоритм с множеством обнаруживает повтор при обработке третьего элемента. Для [3, 1, 2] оба возвращают Ложь.

При ожидаемой постоянной стоимости поиска и добавления в хеш-множество алгоритм выполняется за ожидаемое O(n) времени и требует O(n) дополнительной памяти. Это оценка при предположениях о хешировании, а не гарантия для любого набора данных. Попарный алгоритм в худшем случае выполняет n(n − 1) / 2 сравнений и использует O(1) дополнительной памяти.